Gestion des données

L'ajout

Table des matières

1. Ajout d'une catégorie 1
   1.1. Les contraintes sur les données 1
   1.2. La classe métier Categorie 2
   1.3. Test fonctionnel du contrôleur http://gestion/categorie/ajout/ajax/ajouter.php 3
   1.1. Le contrôleur principal 4
   1.2. Le script index.html 4
   1.3. Le script index.js 5
2. Ajout d'un coureur 6
   2.1. Les contraintes sur les données 6
   2.2. La classe métier Coureur 7
   2.3. Test fonctionnel du contrôleur http://gestion/coureur/ajout/ajax/ajouter.php 8
   2.4. Le contrôleur principal 8
   2.5. Le script index.html 8
   2.6. Le script index.js 9
3. Ajout d'une annonce 10
   3.1. Les contraintes sur les données 10
   3.2. La classe métier Annonce 10
   3.3. Test fonctionnel du contrôleur http://gestion/annonce/ajout/ajax/ajouter.php 10
   3.4. Le contrôleur principal 11
   3.5. Le script index.html 11
   3.6. Le script index.js 11
4. Ajout d'un projet et de ses compétences 12
   4.1. Les contraintes sur les données 12
   4.2. La classe métier Projet (classemetier/projet.php) 12
   4.3. La classe service ProjetServiceL'interface 12
   4.4. Test fonctionnel du contrôleur http://gestion/projet/ajout/ajax/enregistrer.php 12
   4.5. Le contrôleur principal 13
   4.6. Le script index.html 13
   4.7. Le script index.js 13

Guy Verghote

Gestion des données : L'ajout

Objectif de l'activité professionnelle

A la fin de cette activité, vous serez capable :

D'utiliser les classes techniques 'Column' et 'Table' pour réaliser l'ajout de données dans une table De mettre en place une classe métier dérivée de la classe Table permettant d'assurer toutes les opérations sur une table D'assurer l'ensemble des contrôles au niveau de la base de données De mettre en place l'ensemble des contrôles côté serveur De mettre en place côté client l'ensemble des contrôles au niveau de l'interface de saisie De prendre en compte les champs facultatifs lors d'une opération d'ajout De mettre en place une transaction pour réaliser en une seule opération des ajouts dans plusieurs tables

1. Ajout d'une catégorie

1.1. Les contraintes sur les données

Un grand nombre de contraintes sont déjà mises en place sur la table catégorie au niveau de la base de données.

Contrainte	Signification
Primary key	Garantit que chaque valeur dans cette colonne (ou combinaison de colonnes) est **unique** et **non nulle**.
Foreign key	Garantit que les valeurs insérées existent dans la table référencée (intégrité référentielle).
Unique	Garantit que les valeurs dans cette colonne sont uniques (mais peuvent être nulles, sauf si interdit).
Not Null	Empêche la colonne de contenir des **valeurs nulles**.
Check	Impose une condition logique à respecter sur les valeurs d'une colonne.

Si l'un de ces contraintes n'est pas vérifiée, alors le serveur MySQL génère une **exception de classe PDOException** qui sera envoyée en l'absence de try/catch vers le gestionnaire global 'set_exception_handler()' qui va afficher le message d'erreur.
Le message brut retourné est souvent technique et peu compréhensible pour un utilisateur final.

```
[23000][1062] Duplicata du champ 'CA' pour la clef 'categorie.PRIMARY'
[23000][1062] Duplicata du champ 'Cadet' pour la clef 'categorie.nom'
[23000][1452] Cannot add or update a child row: a foreign key constraint fails (`gestion`.`coureur`, CONSTRAINT `coureur_ibfk_2` FOREIGN KEY (`idClub`) REFERENCES `club` (`id`))
```

L'affichage de ces messages dans l'interface utilisateur est à proscrire, pour deux raisons principales :

- Ces messages sont trop techniques pour un utilisateur ;
- Ils peuvent divulguer des informations sensibles sur la structure de la base ou sur son contenu, ce qui pourrait aider un attaquant à affiner une attaque (ex. injection SQL).

La classe Erreur se charge de convertir ces messages en messages plus générique mais compréhensible tout en plaçant le message brut dans le journal d'erreur 'erreur.log'.
Les journaux sont conservés dans le répertoire /log.

Les contraintes sur les tables ne peuvent pas traduire toutes les règles métiers, par exemple des règles spécifiques sur le format de la donnée (le code de la catégorie comporte une ou 2 lettres suivies éventuellement d'un chiffre, des règles nécessitant l'accès à d'autres données de la table ou d'une autre table.

La mise en place d'un trigger peut alors être nécessaire et son instruction signal génère une exception de classe PDOException dont le message d'erreur et clair et affichable à l'utilisateur.

Le déclencheur 'avantAjoutCategorie' permet ici de vérifier l'absence de chevauchement d'âges entre deux catégories.

1.2. La classe métier Categorie

La classe Categorie hérite de la classe Table afin de faciliter la mise en œuvre des opérations de mise à jour sur la classe Table (ajout, modification et suppression).

La classe Table intègre automatiquement les fonctionnalités communes nécessaires à la gestion des données :

- connexion et accès à la base de données ;
- validation des colonnes ;
- contrôle des valeurs obligatoires ;
- contrôle des formats ;
- vérification des contraintes d'unicité ;

Une classe métier qui hérite de Table doit simplement décrire dans sa **méthode configure()** ce qui est spécifique à l'entité métier :

- la table représentée : $this->table = 'categorie' ;
- la clé primaire : $this->primaryKey = 'id' ;
- les colonnes utilisées par l'application (**les colonnes que la couche métier est autorisée à manipuler/valider)** ;

Exemple

```
$this->addColumn('ageMin', new ColumnInt(required: true, min: 4, max: 99));
```

Une colonne est représentée par un objet d'une classe dérivée de la classe Column : ColumnText, ColumnInt, ColumnDate, ColumnMail, ColumnUrl etc.

Chaque objet 'Column' encapsule la valeur de la colonne et l'ensemble des règles de validité que doit respecter cette valeur.

Chaque règle est exprimée par une propriété. Les principales propriétés sont :

- Require : La valeur est obligatoire (valeur par défaut à true)
- **Insertable** : La colonne est utilisable lors d'un ajout (valeur par défaut à true)
- **Uptatable** : La colonne est utilisable lors d'une modification (valeur par défaut à true)
- MinLength : La valeur doit avoir un nombre minimum de caractères
- MaxLength : La valeur doit avoir un nombre maximum de caractères
- Pattern : la valeur doit respecter l'expression régulière

La méthode addColumns ajoute chaque objet Column dans le tableau associatif $columns défini dans la classe de base Table

Important : Certaines colonnes d'une table ne sont pas présente dans ce tableau :

- La clé primaire s'il s'agit d'une valeur auto-incrémentée
- Les colonnes dont la valeur est calculée par un trigger

Concernant la table categorie(id, nom, ageMin, ageMax) les objets suivants sont utilisés :

Colonne	Objet	Propriété	Valeur
id	ColumnText	Require MinLength MaxLength Pattern	true 2 3 ^(M(10|[0-9])|[A-Z]{2})$
nom	ColumnText	Require MinLength MaxLength Pattern	true 5 20 ^[A-Za-z][A-Za-z]*(?: \d{1,2})?$
ageMin	ColumnInt	Require Min Max	true 4 99
ageMax	ColumnInt	Require Min Max	true 4 99

Le **(?:** dans une expression régulière est appelé un groupe non capturant

Les méthodes **before** portent les règles métier applicatives (spécifiques au domaine, au parcours utilisateur, au contexte d'usage) qui ne relèvent pas forcément d'une contrainte SQL.

On peut dupliquer certaines règles SQL dans before pour faire une validation précoce (message plus contextualisé, pas d'aller-retour SQL inutile).

Mais la contrainte en base reste la garantie finale d'intégrité, car elle protège aussi hors application (scripts, imports, autres clients).

Noter qu'en évitant un aller-retour SQL on ne gagne pas énormément de temps, c'est de l'ordre de 5 à 30 ms selon charge/réseau. Donc l'intérêt principal de la validation précoce est souvent la qualité du message et l'UX, plus que la performance brute, sauf traitements en masse.

02 Développement Web> 03 Élément pratiques 08 Comment le mini Framework gère les erreurs PDO provenant de la base de données

1.3. Test fonctionnel du contrôleur http://gestion/categorie/ajout/ajax/ajouter.php

1. Tester sous Insomnia tous les cas possibles

Données transmises manquantes
Données transmises vide
Données transmises avec des valeurs non conforme notamment
- Des valeurs qui ne respectent pas le format attendu
- Un intervalle ageMin ageMax dans le mauvais sens
- Un intervalle générant un chevauchement avec un enregistrement existant

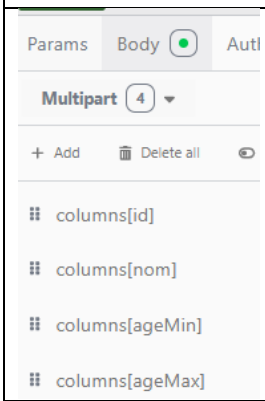
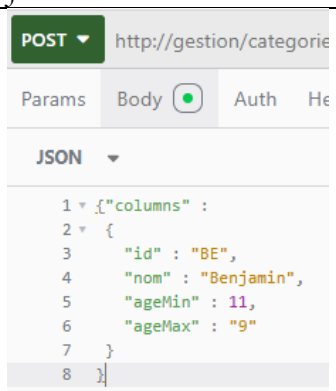
Pour éviter de devoir récupérer le jeton et la variable de session, nous désactivons tous les lignes

```
// Controle::verifier('post', 'jeton');
```

Il faudra bien sur penser à les réactiver à la fin de l'ensemble des tests

Le contrôleur attend ici un passage de paramètre sous la forme d'un tableau (converti en JSON bien sûr)

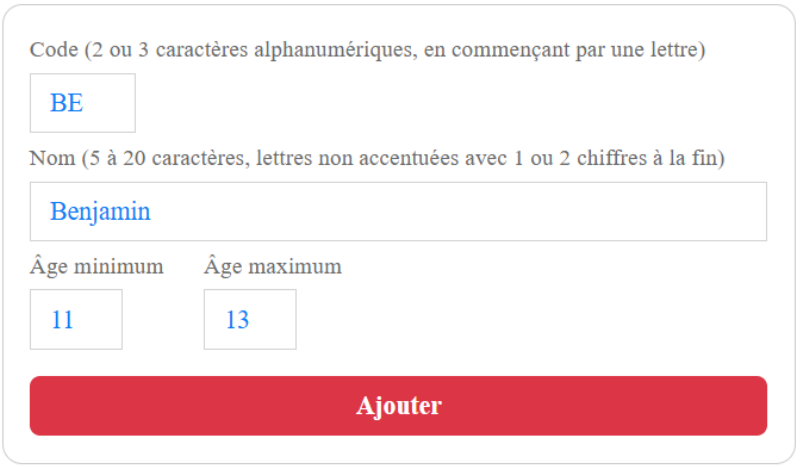
Nous savons que nos méthodes de la classe Requete accepte un passage utilisant le format form_data ou le format JSON

Form-data	JSON
	

1.1. Le contrôleur principal

L'envoi des catégories au client est facultatif et doit être évalué selon le contexte. Les contrôles réalisés côté client ont uniquement pour objectif d'améliorer l'expérience utilisateur et de limiter des requêtes inutiles vers le serveur ; ils ne constituent pas une mesure de sécurité, car ils peuvent toujours être contournés. Par ailleurs, toute donnée transmise au navigateur étant accessible côté client, seules des informations non sensibles doivent être exposées.

1.2. Le script index.html



Pour chaque balise Input, nous retrouvons, exprimée dans ses attributs, les règles de validité qui seront vérifiées côté client cette fois-ci.

Champ	Code	Nom	ageMin - ageMax
Type	Text	text	number
required pattern maxlength minlength	true ^(M(10|[0-9])|[A-Z]{2})$ 3 2	true ^[A-Za-z][A-Za-z]*(?: \d{1,2})?$ 30 3	true 1 (min) 99 (max)

1.3. Le script index.js

Il :

- Contrôle des valeurs saisies à l'aide des propriétés liées aux balises <input>,
- Contrôle les règles métiers spécifiques (unicité, chevauchement …)
- Affiche les éventuels messages d'erreurs
- Réalise l'appel du script /ajax/ajouter.php en lui transmettant les valeurs saisies sur l'interface

La mise en place de ces opérations est facilitée par l'utilisation de fonctions JavaScript développées en collaboration avec les anciennes promotions d'étudiants.
Ces fonctions sont organisées et regroupées dans des bibliothèques de fonctions accessibles depuis le répertoire composant/fonction.

Pour utiliser ces fonctions il suffit de les importer au début du script js:

Il faut commencer par mettre en place les balises qui afficheront les éventuels messages d'erreur en dessous des champs. Cette action est automatisée à l'aide de la fonction **configurerFormulaire()** du module **fonction/formulaire.js** qui va parcourir tous les champs de l'interface pour y placer juste après une balise . La classe 'messageErreur' est générée dynamiquement par la fonction configurerFormulaire().

la fonction filtrerLaSaisie() du même composant permet de filtrer les caractères acceptés dans les champs

Attention : l'utilisation de cette fonction rend impossible le copier/coller dans le champ.

Le bouton 'Ajouter' (événement click) doit commencer par l'effacement des anciens messages d'erreur et une mise en forme des données saisies.

Ensuite il faut vérifier les règles de format (pattern, minlength, maxlength) en appelant la fonction donneesValides du module formulaire.js qui se charge de contrôler chaque champ et d'afficher sous le champ l'erreur rencontrée.
Ici on utilise aussi la fonction spécifique controlerMetier pour réaliser les autres contrôles
En l'absence d'erreur, la fonction ajouter est appelé

La fonction controlerMetier enchaîne les différentes fonctions de contrôle des règles métiers
idExiste() : unicité de l'id
nomExiste() : unicité du nom
intervalleValide() : ageMin < ageMax;
chevaucheUneCategorie()

Langage > JavaScript > Les tableaux

La fonction ajouter appelle le script **ajax/ajouter.php** en lui passant les paramètres attendus dans la propriété columns contenant les propriétés correspondant aux données saisie

L'appel est réalisé par la fonction **appelAjax()** de la bibliothèque fonction/ajax.js. Cette fonction utilise l'API fetch pour réaliser l'appel Ajax et **prend en charge toute la gestion d'erreur.**

Le message d'erreur s'affiche sous le champ concerné ou dans la div 'msg' pour les autres erreurs.
En l'absence de la balise div 'msg', le message est affiché dans une boîte de dialogue.
Coté développeur, il suffit donc d'appeler la fonction appelAjax en lui passant l'url, les données et la fonction à exécuter en cas de succès, ici nous retournons vers la page affichant les catégories afin de voir immédiatement le résultat.

Développement Web > Éléments pratiques > Composant Client > ajax.js

2. Réaliser les tests fonctionnels de ce module

Relancer le script sql/initialisation table/categorie.sql afin de ne pas tenir compte des catégories ajoutées lors de la phase de contrôle sous Insomnia

Tester l'unicité de l'id, l'unicité de nom, la validité de l'intervalle et le chevauchement

2. Ajout d'un coureur

Nous allons étudier un second exemple afin de montrer la prise en compte des champs facultatifs 'email' et 'telephone' (colonne avec la contrainte null) et des contraintes d'intégrité référentielle sur les champs 'idClub' et 'idCategorie'.

2.1. Les contraintes sur les données

Au niveau de la description de la table coureur, nous trouvons les contraintes suivantes
- Unicité : licence, nom + prenom + dateNaissance, email
- Intégrité : idCategorie et idClub
- Null : email et telephone

Dans le trigger 'avantAjoutCoureur' :

Colonne	Contrôle
licence	format : ^[0-9]{6,7}$
nom et prenom	Mise en majuscule : set new.nom = ucase(new.nom); Longueur : entre 3 et 30 Format : ^[A-Za-z](?[A-Za-z])*$
idCategorie	idCategorie est entièrement déduit de la date de naissance du coureur Le trigger détermine année (la saison sportive) : année courante si mois <9 sinon + 1 calcule l'âge du coureur : annee – year(new.dateNaissance) cherche la catégorie correspondant à cet âge : select id from categorie where annee - *year*(new.dateNaissance) between ageMin and AgeMax; récupère l'id de cette catégorie remplit automatiquement new.idCategorie Si aucune catégorie ne correspond la date est forcément invalide

email	Si renseigné, Format : regexp '^[0-9a-zA-Z]([-_\.]?[0-9a-zA-Z])*@[0-9a-zA-Z]([-_\.]?[0-9a-zA-Z])*[\.][a-zA-Z]{2,4}$
telephone	Si renseigné, format : regexp '^0(6|7)[0-9]{8}$'

Au niveau de la classe métier 'Coureur'

Unicité : licence, nom + prenom + dateNaissance, email
Intégrité : idCategorie et idClub

2.2. La classe métier Coureur

Le constructeur permet de définir les colonnes à l'aide d'objets 'Column'. La colonne idCategorie dont la valeur est déduite de la date de naissance est automatiquement calculée par le déclencheur 'avantAjoutCoureur', elle n'est donc pas reprise dans le constructeur

Colonne	Objet	Propriété	Valeur
licence	ColumnText	Require MinLength MaxLength Pattern	true 6 7 ^[0-9]{6,7}$
nom prenom	ColumnText	Require MinLength MaxLength Pattern EnMajuscule SupprimerAccent SupprimerEspaceSuperflu	true 3 30 ^[A-Z](?[A-Z]+)*$ true true true
sexe	ColumnList	Values EnMajuscule	M ou F true
dateNaissance	ColumnDate	Require Min Max	true Categorie::getDateNaissanceMax(); Categorie::getDateNaissanceMin();
idClub	ColumnList	Values	Club::getLesid()
email	ColumnEmail	Require MaxLength	false 100
telephone	ColumnText	Require Pattern MinLength -MaxLength	false ^0(6|7)[0-9]{8} 10

Les objets ColumnList vont permettre de vérifier la valeur transmise. Cet objet contient la liste des valeurs acceptées.
Cette liste peut être alimentée par des valeurs "en dur" comme pour le sexe et la colonne FFA (0/1) ou par des valeurs récupérées dans la base de données comme pour idClub avec la méthode Club::getLesId()

Il est important de bien comprendre le code de cette méthode qui n'utilise pas la classe Select pour récupérer les données car la méthode rows renvoie un tableau de tableau : [['id' => '080004'], ['id' => '080010'], ...]. Or le résultat attendu est ['080004', '080010', …]

La méthode fetchAll avec le paramètre PDO::FETCH_COLUM permet quant à elle d'obtenir ce résultat :

```
public static function getLesId(): array  {
    $sql = "select id from club;"
    $db = Database::getInstance();
    $cmd = $db->query($sql);
    $lesIds = $cmd->fetchAll(PDO::FETCH_COLUMN);
    $cmd->closeCursor();
    return $lesIds;
}
```

Au niveau des contraintes d'unicité elles sont ici au nombre de trois :

- Unicité du numéro de licence
- Unicité de l'email si renseigné
- Unicité du triplet nom + prénom + date de naissance

Nous les retrouvons dans le fichier config/contrainte.php

2.3. Test fonctionnel du contrôleur http://gestion/coureur/ajout/ajax/ajouter.php

3. Tester sous Insomnia tous les cas possibles

Quelques exemples :

Valeur non transmise, format invalide, intégrité référentielle, unicité licence, licence avec des lettres, nom et prénom avec des chiffres, idClub inexistant, ffa avec une valeur erronée, date de naissance au format français, date de naissance trop récente ou trop ancienne, …

2.4. Le contrôleur principal

Il récupère les clubs qui serviront côté lcient à alimenter la zone de liste idClub

Il récupère l'intervalle des dates de naissance acceptées à l'aide des méthodes Categorie::getDateNaissanceMin() et Categorie::getDateNaissanceMax()

Cela permet au client de renseigner dynamiquement les attrributs max et min de la balise

```
<input id="dateNaissance" type="date" required>
```

2.5. Le script index.html

Au niveau de la date de naissance, l'intervalle des dates valides est dynamique. Il est calculé à partir des catégories. La valeur des attributs min et max seront donc récupérés côté serveur, puis envoyés côté client

Les règles de validités sont les suivantes :

Champ	Nom et prénom	Licence	telephone
Type	text	text	number
required	true	true	false
pattern	^[A-Za-z](?[A-Za-z])*$	^[0-9]{6,7}$	^0(6|7)[0-9]{8}$
maxlength	30	6	10
minlength	3	7	10

Champ	sexe
Type	select
Valeur	M et F

Champ	dateNaissance
Type	date
required	
min	Déterminé dynamiquement en fonction de la date courante et des catégories
max	Déterminé dynamiquement en fonction de la date courante et des catégories

Champ	club
Type	select
Valeur	Alimenté dynamiquement à partir de la méthode getListe() de la classe métier Club

Champ	ffa
Type	checkbox

Champ	mail
Type	text (préféré à email pour une expression régulière de vérification plus précise)
pattern	^[0-9a-zA-Z]([-_\.]?[0-9a-zA-Z])*@[0-9a-zA-Z]([-_\.]?[0-9a-zA-Z])*[\.][a-zA-Z]{2,4}$

2.6. Le script index.js

Le script index.php a transmis côté client les clubs et l'intervalle des dates de naissance.

Le script index.js :

- Lance la fonction configurerFormulaire() pour ajouter sous les champs les balises qui afficheront les éventuels messages d'erreur.
- Alimente la zone de liste à l'aide de la variable globale lesClubs
- Utiliser la fonction filtrerLaSaisie() pour limiter les caractères autorisés sur les champs telephone, licence, nom et prénom
- Utilise la fonction configurerDate(input, min, max, valeur) du composant contrôle.js afin de limiter les valeurs possibles sur le champ 'date'
- Réalise l'appel ajax **en tenant compte des données optionnelles**

4. Réaliser les tests fonctionnels de ce module

3. Ajout d'une annonce

Il est souvent nécessaire de permettre aux utilisateurs de mettre en forme le texte saisi depuis un champ de type 'textarea' sans que cela nécessite des connaissances en HTML.

Il existe différents composants permettant de le faire. Ils sont couramment utilisés dans les systèmes de gestion de contenu (CMS), les forums, les blogs et d'autres applications web.

Ils incluent la mise en forme du texte, le support des médias incorporés, la gestion des listes, la création de liens, la gestion des images, etc. Ils offrent en quelque sorte une expérience de type traitement de texte dans un navigateur web.

Les plus connus sont CkEditor et TinyMCE, on peut citer aussi Quill ou Summernote.
Nous utiliserons cette année TinyMCE car la version 4 de CkEditor n'est plus maintenue et sa version 5 n'offre pas les mêmes avantages et ne semble plus gratuite.

3.1. Les contraintes sur les données

La table annonce comprend les colonnes suivantes : id, date, nom, description, actif,dateCreation et dateModification

Nous trouvons 2 contraintes check dont 1 concernent l'ajout (cf. contrainte.php)
Une contrainte sur la date dans le trigger avantAjoutAnnonce

3.2. La classe métier Annonce

Les colonnes suivantes ne sont pas définies car elles sont gérées automatiquement pas le SGBDr

- Le champ id du type auto-incrément..
- Le champ dateCreation ayant une valeur par défaut
- Le champ dateModification

La colonne 'actif' ne doit pas être utilisable lors de l'ajout

Colonne	Objet	Propriété	Valeur
nom	ColumnText	require minLength maxLength pattern	true 8 100 ^[A-Za-zÀÇÈÉÊàáâçèéêëî]([']?[A-Za-zÀÇÈÉÊàáâçèéêëî]+)*$
description	ColumnTextarea	require	true
date	ColumnDate	require min max	true date('Y-m-d') date("Y-m-d", strtotime("+1 year"))
actif	ColumnInt	min max insertable	0 1 false

3.3. Test fonctionnel du contrôleur http://gestion/annonce/ajout/ajax/ajouter.php

5. Tester sous Insomnia tous les cas possibles

Quelques exemples :

Valeur non transmise, format invalide du nom, date inférieure à la date du jour, date au format français, …

3.4. Le contrôleur principal

Il charge le composant tinyMce

3.5. Le script index.html

Il reproduit les contrainte sur le champ nom

3.6. Le script index.js

Il initialise le composant tinyMce et les attributs min, max, value du champ de type date à l'aide de la fonction configurerDate(). La fonction getDateRelative de la bibliothèque date.js permet de fixer les valeurs de min, max et value en fonction des règles établies : entre aujoud'hui et dans un an

Le composant tinyMce est initialisée

```
// Initialisation de TinyMCE
tinymce.init({
    license_key: 'gpl',
    selector: '#description',
    height: 300,
    menubar: false,
    plugins: 'link lists image',
    toolbar: [
        'styles | bold italic underline | forecolor backcolor | fontsizeselect | link | image |bullist numlist'
    ],
    fontsize_formats: '8pt 10pt 12pt 14pt 18pt 24pt 36pt',
});
```

Pour récupérer la valeur du champ 'description' géré par le composant :

```
btnAjouter.onclick = () => {
    // récupération de la valeur du composant tinymce
    description.value = tinymce.get('description').getContent();
    …
};
```

6. Réaliser les tests fonctionnels de ce module

Url de l'image : https://www.klikego.com/documents/?reference=1782196315082-1&index=0

4. Ajout d'un projet et de ses compétences

Cet exemple peut être repris dans le cadre de votre portfolio dans lequel vous devez indiquer l'ensemble des projets réalisés au cours de vos deux années de formation en indiquant les compétences du référentiel mises en œuvre :

La création d'un projet se traduit au niveau de la base de données par l'ajout d'un enregistrement dans la table projet et par l'ajout de plusieurs enregistrements dans la table competenceProjet.

Il n'est donc pas possible d'utiliser le script /ajax/ajouter.php pour 2 raisons :
La table competenceProjet possède une clé compose : idProjet, idCompetence
les deux opérations ne doivent pas être dissociées,

Pour être certains d'exécuter toutes les instructions 'insert' comme un seul bloc, il faut mettre en place une transaction.
Une transaction permet :

d'exécuter un groupe d'opérations SQL comme une seule unité logique ;
d'assurer la cohérence et l'intégrité des données, même en cas d'erreur ou d'interruption inattendue ;
de garantir l'atomicité : soit toutes les opérations sont réalisées avec succès, soit aucune modification n'est appliquée à la base.

En cas de succès, on valide les opérations avec COMMIT.
En cas d'erreur, on annule tout changement avec ROLLBACK.

4.1. Les contraintes sur les données

On peut voir dans le script de création de la table la présence :
- d'une contrainte unique sur le nom du projet,
- d'une contrainte d'unicité sur le coupe idProjet, idCompentence dans la table competenceProjet ainsi que deux contraintes d'intégrité référentielle

Toutes ces contraintes se retrouvent dans le fichier de configuration contrainte.php

4.2. La classe métier Projet (classemetier/projet.php)

La classe Projet prend en charge l'enregistrement d'un projet sans ses compétence.

4.3. La classe service ProjetServiceL'interface

ProjetService est le premier service qui orchestre plusieurs objets métier dans une transaction unique

4.4. Test fonctionnel du contrôleur http://gestion/projet/ajout/ajax/enregistrer.php

Ce contrôleur :
- 1.charge le bootstrap
- 2.lit les paramètres POST contrôlés : nom et lesCompetences (tableau)
- 3. instancie ProjetService
- 4.appelle enregistrer($nom, $lesCompetences)
- 5. renvoie un message de succès JSON

Il délègue la logique métier transactionnelle au service, ce qui garde un contrôleur lisible et maintenable.

7. Tester sous Insomnia tous les cas possibles

4.5. Le contrôleur principal

Il récupère et transmet côté client les compétences du bloc 1

8. Compléter le script index.php

4.6. Le script index.html

Il ne compote que la définition du champ nom et définir un emplacement pour que le script index.js y génère les cases à cocher associer à chaque compétence du bloc 1

4.7. Le script index.js

Le script gère le comportement client :

- 1. récupère les compétences (getData("lesCompetencesBloc1"))
- 2. construit dynamiquement la liste de cases à cocher
- 3. maintient un Set des compétences sélectionnées
- 4. valide le formulaire (donneesValides())
- 5. impose au moins 1 compétence
- 6. envoie l'AJAX vers ajax/enregistrer.php avec : nom et lesCompetences: [...set]
- 7.en succès : redirection vers /projet/liste

Nous avons besoin de conserver dans une structure de donnée les compétences selectionnées (cochées).

Nous pourrions de façon classique utiliser un tableau pour y stocker les numéro des compétences (voir index2.js) mais il peut être plus pratique ici d'utiliser un objet Set qui présente deux avantages importants :

- L'unicité garantie des valeurs : Un Set ne peut jamais contenir de doublons. Même si un élément est ajouté plusieurs fois (ex. via un double-clic ou une erreur d'événement), le Set ne conservera qu'une seule instance de la valeur sans qu'il soit nécessaire d'écrire du code de vérification manuel.
- Des performances optimales pour la recherche et la suppression : Les méthodes comme .has() (pour vérifier la présence d'une valeur) ou .delete() (pour retirer un élément) s'exécutent en temps constant. À l'inverse, avec un tableau classique (Array), la suppression d'un élément nécessite une recherche d'index suivie d'une réorganisation du tableau (splice), des opérations en temps linéaire

Définition d'un objet Set en JavaScript

un Set est une collection d'éléments ordonnés où chaque valeur doit être unique.

Contrairement aux tableaux, les éléments ne sont pas accessibles par un index numérique (pas de monSet[0]).
Les doublons sont automatiquement ignorés lors de l'ajout.
Il est itérable (on peut le parcourir avec une boucle for...of).
Méthodes et propriétés principales :
size Retourne le nombre d'éléments présents dans le Set
add(valeur) Ajoute une valeur (si elle n'existe pas déjà)
delete(valeur) Supprime la valeur spécifiée et renvoie true si elle était présente
has(valeur) Vérifie si une valeur est contenue dans le Set
clear()Supprime tous les éléments du Set.

Si une API ou une fonction exige un tableau (comme lors d'un envoi AJAX), la conversion d'un Set en Array se fait en une ligne grâce à l'opérateur de décomposition (spread operator) :JavaScriptconst tableauCompetences = [...mesCompetencesSet];

L'**opérateur de décomposition** (*spread operator* en anglais), représenté par **trois points de suspension (...)**, permet d'**éclater** une structure de données itérable (un tableau, un objet ou un Set) en plusieurs éléments individuels.

9. Réaliser les tests fonctionnels de ce module